
FinWise AI Guide and Interview Questions

This file contains common questions a project guide, external examiner, or interviewer may ask about FinWise AI. The answers are written in simple language so they can be used directly for viva preparation.

A. Basic Project Questions

1. What is your project?

FinWise AI is a personal finance management system. It helps users track accounts, income, expenses, transfers, budgets, savings goals, ledger entries, analytics, reports, bank statement imports, and offline finance guidance.

2. Why did you choose this project?

I chose this project because personal money tracking is a real daily problem. Many people do not know where their money is going, how much they saved, or whether they are overspending. This project solves that in a simple and practical way.

3. What problem does your project solve?

It solves the problem of scattered financial records. Instead of checking many bank apps, notes, and spreadsheets, the user can manage income, expenses, accounts, loans, credit cards, goals, and monthly analytics in one place.

4. Who can use this project?

Students, working people, small families, freelancers, and anyone who wants to track personal money can use this project.

5. What is the main objective?

The main objective is to give a simple offline personal finance system that tracks money clearly and gives explainable AI-based finance help.

6. Is this a real-time finance application?

It is not connected to live bank APIs. It is a local finance tracker where the user enters data manually or imports bank statements.

7. Is it a web application?

Yes. It is a local web application. The frontend runs in the browser, the backend runs using Flask, and data is stored in MySQL.

8. Can it run offline?

Yes. After installation, the project can run offline on the same computer. It does not need an online AI API for the finance chat.

B. Features Questions

9. What are the main modules?

The main modules are Login, Profile, Dashboard, Accounts, Transactions, Ledger, Budgets, Savings Goals, Analytics, Reports, Finance Chat, Bank Statement Import, and Smart Tools.

10. What does the dashboard show?

The dashboard shows Total Money, Balance Money, Income, Expense, Savings, monthly overview, recent transactions, and quick actions.

11. What is Total Money?

Total Money means balance money plus money saved in goals.

$\text{Total Money} = \text{Balance Money} + \text{Goal Savings}$

12. What is Balance Money?

Balance Money is the money currently available in accounts.

13. What is Savings in your project?

Savings means money moved into savings goals. It is tracked separately from normal expenses.

14. Why is goal saving not counted as an expense?

Because the user still owns the money. It is only moved from a bank account into a savings goal. It is not spent on food, rent, bills, or shopping.

15. What account types are supported?

Bank, Cash, UPI Wallet, Credit Card, Loan, and Investment.

16. How do you track a loan?

A loan can be added as a Loan account. This makes the liability visible in the same finance system.

17. How do you track a credit card?

A credit card can be added as a Credit Card account. Credit card spending can be tracked separately from normal bank accounts.

18. What are the transaction types?

Income, Expense, and Transfer.

19. What is a transfer?

A transfer is money moved from one account owned by the user to another account owned by the same user. It is not income and not expense.

20. Why do you need transfer?

Without transfer, moving money between two own accounts may be wrongly counted as income or expense. Transfer keeps tracking correct.

C. Technical Stack Questions

21. Which frontend technology is used?

React is used for the frontend. Vite is used for building the frontend.

22. Which backend technology is used?

Flask is used for the backend API.

23. Which database is used?

MySQL is used to store user, account, transaction, budget, goal, report, import, and preference data.

24. Why did you use React?

React makes it easier to build reusable components and dynamic pages like dashboard, charts, forms, and tables.

25. Why did you use Flask?

Flask is simple, lightweight, and suitable for building REST APIs for a project like this.

26. Why did you use MySQL?

MySQL is reliable for structured data. Finance records have clear tables and relationships, so MySQL is a good fit.

27. What is the role of Chart.js?

Chart.js is used to show analytics charts such as category-wise expense breakdown.

28. What libraries are used for statement import?

openpyxl and xlrd are used for Excel files. pdfplumber is used for text-based PDF files. CSV files are handled using normal CSV parsing.

D. AI and Algorithm Questions

29. Where is AI used in your project?

AI is used in transaction category prediction and offline finance chat guidance.

30. What algorithm is used for category prediction?

Multinomial Naive Bayes is used for category prediction.

31. Why did you choose Naive Bayes?

Naive Bayes is simple, fast, explainable, and works well for text classification problems like predicting category from transaction descriptions.

32. How does Naive Bayes work here?

It reads words in the transaction description and calculates which category is most likely based on previous examples and finance keywords.

33. Give an example of category prediction.

```
"salary july" -> Salary  
"swiggy dinner" -> Food  
"electricity bill" -> Bills
```

34. What is the formula idea behind Naive Bayes?

$P(\text{category} \mid \text{words}) \propto P(\text{category}) * P(\text{word1} \mid \text{category}) * P(\text{word2} \mid \text{category})$

The category with the highest probability is selected.

35. Is the AI chat an online chatbot?

No. It is an offline rule-based finance assistant. It does not call ChatGPT or any cloud AI API.

36. Why is offline AI useful?

It keeps the project simple, private, and free to run. It also avoids dependency on internet and paid APIs.

37. What is rule-based intent matching?

It means the system checks keywords in the user's question and returns a matching answer from local finance rules and project help topics.

38. What algorithms are used apart from Naive Bayes?

Ledger running balance, duplicate detection, budget utilization, savings rate, goal progress, month-wise analytics, and financial health rules.

39. Explain the ledger algorithm.

The ledger starts with opening balance and updates balance after each debit or credit.

$$\text{Running Balance} = \text{Opening Balance} + \text{Credits} - \text{Debits}$$

40. Explain duplicate detection.

Before importing a statement, the system checks whether a similar transaction already exists for the same account, date, amount, type, and description.

41. Explain budget utilization.

$$\text{Budget Usage Percent} = \text{Actual Expense} / \text{Budget Limit} * 100$$

It shows how much of the budget is already used.

42. Explain goal progress.

$$\text{Goal Progress Percent} = \text{Saved Amount} / \text{Target Amount} * 100$$

It shows how close the user is to the target.

43. Explain savings rate.

$$\text{Savings Rate} = (\text{Income} - \text{Expense}) / \text{Income} * 100$$

It shows how much of income remains after expenses.

44. Explain month-wise analytics.

The system filters transactions by selected month and calculates income, expense, net flow, saved-to-goal amount, top category, and chart values.

45. What makes the algorithms explainable?

The calculations are simple and can be shown using formulas. The user and guide can understand how each result is produced.

E. Database Questions

46. What are the main database tables?

The main tables include users, accounts, transactions, budgets, savings_goals, goal_contributions, statement_imports, recurring_transactions, user_preferences, password_reset_tokens, and email_verification_tokens.

47. What is stored in the users table?

User name, email, hashed password, verification status, and login-related fields.

48. What is stored in the accounts table?

Account name, account type, subtype, balance, bank details, credit limit, interest rate, and related account information.

49. What is stored in the transactions table?

Transaction type, category, amount, description, date, account ID, transfer account ID, and import reference.

50. What is stored in savings_goals?

Goal name, target amount, saved amount, target date, and user ID.

51. Why do you need goal_contributions?

It stores each deposit or withdrawal from a savings goal, so the history of goal movement is clear.

52. Why do you need statement_imports?

It stores import history, file name, account, number of rows, and imported transaction details.

53. What is a foreign key?

A foreign key links one table to another. For example, transactions are linked to a user and an account.

54. How do you prevent one user from seeing another user's data?

Most queries filter data using the logged-in user's ID.

55. Why is MySQL suitable here?

The project has structured records and relationships, so relational tables are useful.

F. Security Questions

56. Are passwords stored directly?

No. Passwords are stored as hashes.

57. What is password hashing?

Hashing converts the password into a one-way encrypted-looking value. The original password is not stored.

58. What is used for login session?

The app uses JWT with httpOnly cookies.

59. Why are httpOnly cookies useful?

They reduce the risk of JavaScript stealing login tokens from the browser.

60. Are reset tokens visible?

No. Reset and verification tokens are hidden by default.

61. Where should database passwords be stored?

In a local `.env` file, not directly in source code.

62. Is this ready for public deployment?

It is mainly a local academic project. For public deployment, stronger production configuration, HTTPS, email provider setup, and security review are needed.

G. Statement Import Questions

63. What file formats can be imported?

CSV, Excel, and text-based PDF files.

64. What happens before import?

The system shows a preview. The user can check and correct rows before saving.

65. Why is preview important?

Bank statements may have different formats, so preview lets the user correct mistakes before data enters the database.

66. Can scanned PDF statements be imported?

Not reliably. The current PDF import is for text-based PDFs. Scanned PDFs need OCR support.

67. What if the same statement is imported twice?

Duplicate detection marks already existing transactions so repeated import can be avoided.

68. Can imported rows be undone?

The app keeps statement import history, which helps manage imported rows.

H. Ledger and Accounting Questions

69. What is a ledger?

A ledger is an account-wise record of money movement using debit, credit, and running balance.

70. Why did you add ledger?

Ledger makes the project stronger because it follows accounting-style tracking and clearly shows how each transaction changes balance.

71. What is debit?

In this project, debit means money going out of an account.

72. What is credit?

In this project, credit means money coming into an account.

73. How is income shown in ledger?

Income is shown as a credit.

74. How is expense shown in ledger?

Expense is shown as a debit.

75. How is transfer shown in ledger?

Transfer is shown as debit from the source account and credit to the destination account.

76. Why is ledger useful for debugging finance data?

It shows every account movement step by step, so wrong balances are easier to find.

I. Analytics and Finance Questions

77. What does analytics show?

Analytics shows monthly income, expense, net flow, saved-to-goal amount, top category, charts, and month transactions.

78. How can the user check last month and this month?

The Analytics page has a month selector. The user can choose the required month and compare totals.

79. What is net flow?

$\text{Net Flow} = \text{Income} - \text{Expense}$

80. What is top category?

Top category is the category where the user spent the highest amount in the selected month.

81. Why are charts useful?

Charts make spending patterns easier to understand than only reading table rows.

82. What is budget risk?

Budget risk means spending in a category is close to or above the budget limit.

83. What is financial health?

Financial health means whether income, expense, savings, budget, and balance are in a good condition.

J. Smart Tools and Automation Questions

84. What is automation in this project?

Automation means saved auto entries for repeated transactions like salary, rent, EMI, bills, and subscriptions.

85. Why is auto entry useful?

It saves time because the user does not need to manually enter the same repeated transaction again and again.

86. Give examples of auto entries.

Salary, rent, electricity bill, EMI, internet bill, subscription, and insurance payment.

87. Is automation the same as bank sync?

No. Automation is local recurring entry setup. Bank sync would need real bank API integration.

K. Frontend Questions

88. How is navigation handled?

The dashboard has sidebar links to pages such as Accounts, Transactions, Ledger, Budget, Goals, Analytics, Reports, Finance Chat, Smart Tools, and Profile.

89. How is data displayed?

Data is displayed using cards, forms, tables, charts, and summary sections.

90. How is dark mode handled?

Theme preference is available in the profile area. The user can choose light, dark, or system mode.

91. Why use cards in dashboard?

Cards make important values like income, expense, and balance easy to see quickly.

92. Why use tables?

Tables are useful for transactions, ledger entries, imports, budgets, and history because they show structured records clearly.

L. Backend Questions

93. What is the role of Flask API?

The Flask API receives requests from the frontend, checks login, applies finance logic, reads or writes MySQL data, and sends JSON responses.

94. What is JSON?

JSON is a common data format used to send data between frontend and backend.

95. What is an API route?

An API route is a backend URL that performs a specific action, such as loading accounts or saving a transaction.

96. What happens when a transaction is added?

The backend saves the transaction and updates the related account balance based on transaction type.

97. What happens when a transfer is added?

The source account balance decreases and the destination account balance increases.

98. What happens when a goal contribution is added?

For a deposit, account balance decreases and goal saved amount increases. For a withdrawal, account balance increases and goal saved amount decreases.

M. Testing Questions

99. How did you test the project?

The project has backend safety tests and frontend build checks. Manual testing is also done by running the app and checking major flows.

100. What automated tests are included?

Tests check secret safety, hidden reset tokens, frontend popup removal, statement import safety, pinned dependencies, offline chat topics, family removal, and ledger connection.

101. How do you test frontend build?

By running the frontend build command and checking that it compiles successfully.

102. How do you test the database?

By running the app, logging in, adding accounts, adding transactions, checking balances, and verifying records in the UI.

103. How do you test import?

By uploading sample bank statements, reviewing preview rows, confirming import, and checking if transactions appear correctly.

104. How do you test ledger?

By adding income, expense, and transfer transactions, then checking debit, credit, and running balance.

N. Error and Limitation Questions

105. What are the limitations of your project?

It does not use real bank API sync, scanned PDF OCR, mobile app support, or online large language model AI.

106. What happens if MySQL is not running?

The app cannot load database data until MySQL is started and configured.

107. What happens if dependencies are missing?

The app may fail to start. The user must install required Python and Node dependencies.

108. Why did pandas get removed from import processing?

Because some systems block heavy pandas or numpy DLL files. The project now uses lighter libraries for CSV, Excel, and PDF statement import.

109. Is the project scalable?

It is suitable for a local academic project. For many users in production, deployment, security, database indexing, monitoring, and cloud hosting would need improvement.

110. What is future scope?

Future scope includes bank API sync, OCR for scanned statements, mobile app, advanced forecasting, EMI calculations, and optional online AI integration.

O. Interview-Style Technical Questions

111. Explain the project architecture.

The architecture has three layers: React frontend, Flask backend, and MySQL database. The frontend sends API requests to Flask. Flask applies logic and reads or writes data in MySQL.

112. What is REST API?

REST API is a way for frontend and backend to communicate using HTTP methods such as GET, POST, PUT, and DELETE.

113. What is CRUD?

CRUD means Create, Read, Update, and Delete. The project uses CRUD for accounts, transactions, budgets, goals, and profile.

114. What is authentication?

Authentication checks whether the user is logged in and allowed to access the system.

115. What is authorization?

Authorization checks whether the logged-in user has permission to access a specific data record.

116. What is input validation?

Input validation checks user input before saving it, such as amount, date, account, and transaction type.

117. Why should amount be validated?

Because invalid or negative amounts can create wrong finance records.

118. Why should date be validated?

Because analytics and monthly reports depend on correct transaction dates.

119. Why should duplicate records be prevented?

Duplicate records make income, expense, balance, and analytics wrong.

120. What is the difference between frontend and backend?

Frontend is what the user sees in the browser. Backend handles data processing, login, finance logic, and database operations.

121. What is the difference between local storage and httpOnly cookies?

Local storage can be accessed by browser JavaScript. httpOnly cookies cannot be accessed by JavaScript, so they are safer for login sessions.

122. What is SQL injection?

SQL injection is an attack where unsafe user input is used to change a database query. Parameterized queries help prevent it.

123. How does your project reduce SQL injection risk?

The backend uses parameterized queries for database operations.

124. What is data normalization?

Data normalization means organizing data into related tables to reduce duplication and improve consistency.

125. Why use separate tables for accounts and transactions?

One account can have many transactions. Keeping them separate creates a clean one-to-many relationship.

P. Project Defense Questions

126. Your guide says the project is simple. How will you defend it?

The UI is simple for users, but internally it includes multiple finance modules, offline AI, Naive Bayes category prediction, ledger accounting, duplicate detection, analytics, budgets, goals, and secure login.

127. How will you prove algorithms are used?

I can show the category prediction, ledger formula, budget usage formula, savings rate formula, goal progress formula, duplicate detection, and month-wise analytics logic.

128. Why not use a complex AI model?

For a personal finance academic project, explainable local AI is better. It is private, offline, free, and easier to justify.

129. What is unique about your project?

It combines normal finance tracking with ledger accounting, statement import, duplicate detection, offline AI chat, and Naive Bayes category prediction.

130. What is the most important feature?

The most important feature is accurate money tracking through accounts, transactions, transfers, and ledger.

131. What is the hardest part of the project?

Handling different finance flows correctly, especially transfers, goal savings, statement import duplicates, and account balances.

132. What did you learn?

I learned frontend development, backend APIs, database design, authentication, finance logic, algorithms, testing, and user-friendly project design.

133. What would you improve next?

I would add OCR for scanned statements, bank API sync, mobile app support, better forecasting, and detailed credit card/loan calculations.

Q. Quick One-Line Answers

134. Is transfer income?

No. Transfer only moves money between own accounts.

135. Is saving to goal an expense?

No. It is tracked as savings because the user still owns the money.

136. Is credit card an asset?

No. Credit card balance is a liability.

137. Is loan an asset?

No. Loan is a liability.

138. Is bank account an asset?

Yes. Bank balance is an asset.

139. Is the AI online?

No. It is offline and local.

140. Is the project free to run?

Yes, after installing required local software.

141. What is the database?

MySQL.

142. What is the backend?

Flask.

143. What is the frontend?

React.

144. What is the category prediction algorithm?

Multinomial Naive Bayes.

145. What is the ledger formula?

Opening balance plus credits minus debits.

146. What is net flow?

Income minus expense.

147. What is budget usage?

Actual expense divided by budget limit, multiplied by 100.

148. What is goal progress?

Saved amount divided by target amount, multiplied by 100.

149. What is duplicate detection used for?

To stop repeated bank statement imports from creating duplicate transactions.

150. What is the final conclusion?

FinWise AI is a practical offline finance management system with explainable algorithms and useful personal money tracking features.